

MATRICE JACOBIENNE

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

4 août 2026

Table des matières

I	Cours Magistral	2
1	Jalon 46 : Matrice jacobienne et Règle de la chaîne	2
1.1	Genèse et Intuition Géométrique	2
1.2	Définitions et Théorèmes	2
1.2.1	La Matrice Jacobienne	2
1.2.2	Règle de la Chaîne (Chain Rule)	3
1.3	Démonstrations	3
1.3.1	Preuve de la Règle de la Chaîne	3
1.4	Application en IA : Rétropropagation	4
II	Exercices d'Application et de Concours	5
III	Travaux Pratiques et Simulations Algorithmiques	15

Première partie

Cours Magistral

1 Jalon 46 : Matrice jacobienne et Règle de la chaîne

1.1 Genèse et Intuition Géométrique

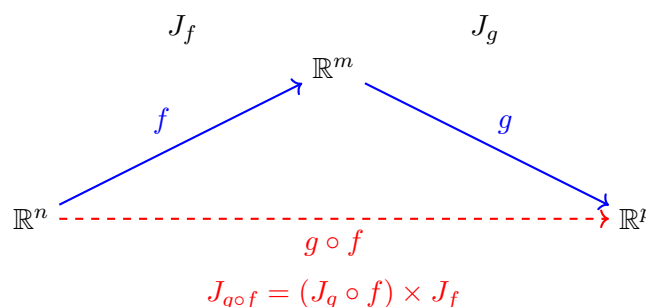


FIGURE 1 – Règle de la chaîne pour les différentielles et Jacobiennes

La matrice jacobienne généralise la notion de dérivée aux fonctions vectorielles de plusieurs variables. Historiquement, Carl Gustav Jacob Jacobi a formalisé ce concept au XIX^e siècle pour étudier les changements de variables dans les intégrales multiples et les équations différentielles.

Si l'on considère une transformation $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, elle déforme un domaine de l'espace de départ. Localement, au voisinage d'un point a , cette déformation non-linéaire peut être approchée par une transformation affine. La partie linéaire de cette transformation est précisément représentée par la matrice jacobienne $J_f(a)$. Elle indique comment une petite perturbation δx autour de a se traduit par une perturbation $\delta y \approx J_f(a)\delta x$ autour de $f(a)$.

En apprentissage profond (Deep Learning), la règle de la chaîne (Chain Rule) est le cœur du calcul du gradient par rétropropagation (backpropagation). Un réseau de neurones n'est autre qu'une gigantesque composition de fonctions, et la dérivée de cette composition requiert le produit ordonné des matrices jacobienes de chaque couche.

1.2 Définitions et Théorèmes

1.2.1 La Matrice Jacobienne

Soit U un ouvert de $\mathbb{R}^n$ et $f : U \rightarrow \mathbb{R}^m$ une application différentiable en un point $a \in U$. On note les composantes de f par $f(x) = (f_1(x), \dots, f_m(x))^T$, où chaque $f_i : U \rightarrow \mathbb{R}$.

Définition : La matrice jacobienne de f en a , notée $J_f(a)$ ou $\text{Mat}(df_a)$, est la matrice de taille $m \times n$ dont le coefficient à la i -ème ligne et j -ème colonne est la dérivée partielle de f_i par rapport à x_j , évaluée en a :

$$J_f(a) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(a) & \frac{\partial f_1}{\partial x_2}(a) & \dots & \frac{\partial f_1}{\partial x_n}(a) \\ \frac{\partial f_2}{\partial x_1}(a) & \frac{\partial f_2}{\partial x_2}(a) & \dots & \frac{\partial f_2}{\partial x_n}(a) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(a) & \frac{\partial f_m}{\partial x_2}(a) & \dots & \frac{\partial f_m}{\partial x_n}(a) \end{pmatrix}$$

Notations et Précisions : - m : dimension de l'espace d'arrivée (nombre de lignes). - n : dimension de l'espace de départ (nombre de colonnes). - La ligne i correspond au vecteur gradient $\nabla f_i(a)^T$.

Exemples : Soit $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ définie par $f(x, y) = (x^2y, \sin(x+y), e^{xy})^T$.

$$J_f(x, y) = \begin{pmatrix} 2xy & x^2 \\ \cos(x+y) & \cos(x+y) \\ ye^{xy} & xe^{xy} \end{pmatrix}$$

$$\text{En } a = (0, \pi), J_f(0, \pi) = \begin{pmatrix} 0 & 0 \\ -1 & -1 \\ \pi & 0 \end{pmatrix}.$$

1.2.2 Règle de la Chaîne (Chain Rule)

Soit U un ouvert de $\mathbb{R}^n$, V un ouvert de $\mathbb{R}^p$. Soit $f : U \rightarrow V$ et $g : V \rightarrow \mathbb{R}^m$.

Théorème : Si f est différentiable en $a \in U$ et g est différentiable en $f(a) \in V$, alors la fonction composée $h = g \circ f$ est différentiable en a . De plus, sa différentielle est la composition des différentielles :

$$d(g \circ f)_a = dg_{f(a)} \circ df_a$$

Matriciellement, cela se traduit par le produit des matrices jacobiniennes :

$$J_{g \circ f}(a) = J_g(f(a)) \times J_f(a)$$

Notations et Précisions : - **Compatibilité des tailles** : $J_g(f(a))$ est de taille $m \times p$ et $J_f(a)$ est de taille $p \times n$. Le produit est bien de taille $m \times n$, ce qui correspond à la matrice jacobienne de $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$. - L'ordre de multiplication est strictement de la gauche vers la droite par rapport à la composition (de l'extérieur vers l'intérieur).

Exemples : Si $f(t) = (t, t^2)^T$ (courbe de $\mathbb{R} \rightarrow \mathbb{R}^2$) et $g(x, y) = x^2 + y^2$ (fonction de $\mathbb{R}^2 \rightarrow \mathbb{R}$). $J_f(t) = \begin{pmatrix} 1 \\ 2t \end{pmatrix}$. $J_g(x, y) = (2x \ 2y)$. Alors $J_{g \circ f}(t) = J_g(f(t))J_f(t) = (2t \ 2t^2) \begin{pmatrix} 1 \\ 2t \end{pmatrix} = 2t + 4t^3$.

On vérifie : $h(t) = t^2 + t^4$, d'où $h'(t) = 2t + 4t^3$.

Cas particuliers et contre-exemples : Si f ou g admet des dérivées partielles mais n'est pas différentiable, la formule de la chaîne peut tomber en défaut. La différentiabilité (existence d'un plan tangent approchant) est cruciale, la simple dérivabilité directionnelle ne suffit pas.

1.3 Démonstrations

1.3.1 Preuve de la Règle de la Chaîne

Objectif : Démontrer que $d(g \circ f)_a = dg_{f(a)} \circ df_a$.

Preuve pas-à-pas : 1. Par définition de la différentiabilité de f en a : Pour tout accroissement $h \in \mathbb{R}^n$,

$$f(a+h) = f(a) + df_a(h) + \|h\|\epsilon_1(h)$$

avec $\lim_{h \rightarrow 0} \epsilon_1(h) = 0$. Posons $k(h) = df_a(h) + \|h\|\epsilon_1(h)$. Notons que $f(a+h) = f(a) + k(h)$ et que $k(h) \rightarrow 0$ lorsque $h \rightarrow 0$.

2. Par définition de la différentiabilité de g en $b = f(a)$: Pour tout accroissement $k \in \mathbb{R}^p$,

$$g(b+k) = g(b) + dg_b(k) + \|k\|\epsilon_2(k)$$

avec $\lim_{k \rightarrow 0} \epsilon_2(k) = 0$. On pose conventionnellement $\epsilon_2(0) = 0$ pour prolonger par continuité.

3. Substituons $k(h)$ dans le développement de g :

$$(g \circ f)(a+h) = g(f(a+h)) = g(f(a) + k(h))$$

$$(g \circ f)(a + h) = g(f(a)) + dg_{f(a)}(k(h)) + \|k(h)\|\epsilon_2(k(h))$$

4. Exploitions la linéarité de $dg_{f(a)}$ pour développer $dg_{f(a)}(k(h))$:

$$dg_{f(a)}(k(h)) = dg_{f(a)}(df_a(h) + \|h\|\epsilon_1(h)) = dg_{f(a)}(df_a(h)) + \|h\|dg_{f(a)}(\epsilon_1(h))$$

5. Regroupons les termes pour former le développement limité de $g \circ f$:

$$(g \circ f)(a + h) = (g \circ f)(a) + (dg_{f(a)} \circ df_a)(h) + R(h)$$

où le reste est $R(h) = \|h\|dg_{f(a)}(\epsilon_1(h)) + \|k(h)\|\epsilon_2(k(h))$.

6. Montrons que $R(h) = o(\|h\|)$ (c'est-à-dire $\lim_{h \rightarrow 0} \frac{R(h)}{\|h\|} = 0$) :

$$\frac{R(h)}{\|h\|} = dg_{f(a)}(\epsilon_1(h)) + \frac{\|k(h)\|}{\|h\|}\epsilon_2(k(h))$$

- Comme $\epsilon_1(h) \rightarrow 0$ et que l'application linéaire $dg_{f(a)}$ est continue (en dimension finie), $dg_{f(a)}(\epsilon_1(h)) \rightarrow 0$. - Majorons la norme de l'accroissement $k(h)$:

$$\|k(h)\| = \|df_a(h) + \|h\|\epsilon_1(h)\| \leq \|df_a\| \cdot \|h\| + \|h\|\|\epsilon_1(h)\| = \|h\|(\|df_a\| + \|\epsilon_1(h)\|)$$

Ainsi, le ratio $\frac{\|k(h)\|}{\|h\|}$ est borné au voisinage de 0. - Puisque $k(h) \rightarrow 0$ lorsque $h \rightarrow 0$, on a $\epsilon_2(k(h)) \rightarrow 0$. Le produit d'une fonction bornée et d'une fonction tendant vers 0 tend vers 0.

7. Conclusion :

$$(g \circ f)(a + h) = (g \circ f)(a) + (dg_{f(a)} \circ df_a)(h) + o(\|h\|)$$

L'application $L = dg_{f(a)} \circ df_a$ est linéaire continue (composée de deux linéaires continues), elle satisfait l'équation de différentiabilité. Par unicité, la différentielle de la composée est la composée des différentielles. ■

1.4 Application en IA : Rétropropagation

La Rétropropagation (Backpropagation) repose fondamentalement sur le calcul vectoriel de la Règle de la Chaîne. Un réseau de neurones multicouche est une fonction :

$$F(x) = (\sigma_L \circ W_L \circ \sigma_{L-1} \circ W_{L-1} \cdots \circ \sigma_1 \circ W_1)(x)$$

Où W_l est l'application linéaire (poids de la couche) et σ_l la fonction d'activation non linéaire.

Pour optimiser les poids W_l , on doit évaluer le gradient de la fonction de perte $L(F(x), y)$ par rapport à ces paramètres. La Jacobienne totale s'écrit comme un immense produit matriciel de Jacobiennes locales. Pour éviter de calculer et stocker des matrices géantes, l'algorithme procède à des **Vecteur-Jacobian Products (VJP)**. On commence par la fin (la perte est un scalaire, son gradient est un vecteur) et on multiplie itérativement ce vecteur par la transposée de la jacobienne de la couche précédente :

$$\delta_l = (J_{\sigma_l})^T \cdot W_{l+1}^T \cdot \delta_{l+1}$$

L'élégance de cette propagation arrière des erreurs tient entièrement à l'associativité du produit matriciel issu de la différentielle des fonctions composées.

Deuxième partie

Exercices d'Application et de Concours

Exercice 1 : Calcul de Jacobienne basique

Énoncé

Soit la fonction $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ définie par :

$$f(x, y) = \begin{pmatrix} x^2 y \\ e^{x+y} \\ \cos(xy) \end{pmatrix}$$

Calculer la matrice jacobienne $J_f(x, y)$ en tout point $(x, y) \in \mathbb{R}^2$.

Correction Détaillée

Soient les composantes $f_1(x, y) = x^2 y$, $f_2(x, y) = e^{x+y}$ et $f_3(x, y) = \cos(xy)$. Les dérivées partielles par rapport à x (première colonne) : - $\frac{\partial f_1}{\partial x} = 2xy$ - $\frac{\partial f_2}{\partial x} = e^{x+y}$ - $\frac{\partial f_3}{\partial x} = -y \sin(xy)$

Les dérivées partielles par rapport à y (deuxième colonne) : - $\frac{\partial f_1}{\partial y} = x^2$ - $\frac{\partial f_2}{\partial y} = e^{x+y}$ - $\frac{\partial f_3}{\partial y} = -x \sin(xy)$

La matrice jacobienne est donc de dimension 3×2 :

$$J_f(x, y) = \begin{pmatrix} 2xy & x^2 \\ e^{x+y} & e^{x+y} \\ -y \sin(xy) & -x \sin(xy) \end{pmatrix}$$

■

Exercice 2 : Jacobienne du changement de coordonnées cylindriques



Énoncé

Soit la fonction de changement en coordonnées cylindriques $f : \mathbb{R}^+ \times [0, 2\pi[\times \mathbb{R} \rightarrow \mathbb{R}^3$:

$$f(r, \theta, z) = \begin{pmatrix} r \cos \theta \\ r \sin \theta \\ z \end{pmatrix}$$

Calculer $J_f(r, \theta, z)$ et son déterminant.

Correction Détaillée

1. **Dérivées partielles** : - Par rapport à r : $\cos \theta, \sin \theta, 0$ - Par rapport à θ : $-r \sin \theta, r \cos \theta, 0$ - Par rapport à z : $0, 0, 1$

2. **Matrice Jacobienne** :

$$J_f(r, \theta, z) = \begin{pmatrix} \cos \theta & -r \sin \theta & 0 \\ \sin \theta & r \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

3. **Déterminant (Le Jacobien)** : En développant par rapport à la dernière ligne :

$$\det(J_f) = 1 \cdot \det \begin{pmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{pmatrix}$$

$$\det(J_f) = r \cos^2 \theta - (-r \sin^2 \theta) = r(\cos^2 \theta + \sin^2 \theta) = r$$



Exercice 3 : Jacobienne du changement de coordonnées sphériques

★★

Énoncé

Soit la fonction $f : \mathbb{R}^+ \times [0, 2\pi[\times [0, \pi] \rightarrow \mathbb{R}^3$:

$$f(r, \theta, \phi) = \begin{pmatrix} r \cos \theta \sin \phi \\ r \sin \theta \sin \phi \\ r \cos \phi \end{pmatrix}$$

Calculer le déterminant jacobien de f .

Correction Détaillée

1. Construction de la matrice :

$$J_f = \begin{pmatrix} \cos \theta \sin \phi & -r \sin \theta \sin \phi & r \cos \theta \cos \phi \\ \sin \theta \sin \phi & r \cos \theta \sin \phi & r \sin \theta \cos \phi \\ \cos \phi & 0 & -r \sin \phi \end{pmatrix}$$

2. **Calcul du déterminant** : Développons par rapport à la deuxième colonne (ou troisième ligne) : $\det(J_f) = \cos \phi (-r \sin \theta \sin \phi \cdot r \sin \theta \cos \phi - r \cos \theta \sin \phi \cdot r \cos \theta \cos \phi) - r \sin \phi (\cos \theta \sin \phi \cdot r \cos \theta \sin \phi - (-r \sin \theta \sin \phi) \cdot \sin \theta \sin \phi)$ Simplifions : $\det(J_f) = \cos \phi (-r^2 \sin \phi \cos \phi (\sin^2 \theta + \cos^2 \theta)) - r \sin \phi (r \sin^2 \phi (\cos^2 \theta + \sin^2 \theta))$ $\det(J_f) = -r^2 \sin \phi \cos^2 \phi - r^2 \sin^3 \phi$ $\det(J_f) = -r^2 \sin \phi (\cos^2 \phi + \sin^2 \phi) = -r^2 \sin \phi$. *Remarque : la valeur absolue donne $r^2 \sin \phi$, l'élément de volume standard en géométrie.* ■

Exercice 4 : Règle de la chaîne dimension $1 \rightarrow 2 \rightarrow 1$ ★★

Énoncé

Soit $f : \mathbb{R} \rightarrow \mathbb{R}^2$ définie par $f(t) = (t^2, e^t)^T$ et $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par $g(x, y) = xy^2$. Soit $h = g \circ f$. Calculer $h'(t)$ de deux manières différentes : 1) Par composition explicite puis dérivation classique. 2) En utilisant le produit des matrices jacobiniennes.

Correction Détaillée

Méthode 1 : Composition explicite $h(t) = g(f(t)) = g(t^2, e^t) = (t^2)(e^t)^2 = t^2e^{2t}$. La dérivée (produit) : $h'(t) = 2te^{2t} + t^2(2e^{2t}) = 2te^{2t}(1 + t)$.

Méthode 2 : Produit des Jacobiniennes - $J_f(t) = \begin{pmatrix} 2t \\ e^t \end{pmatrix}$ - $J_g(x, y) = (y^2 \quad 2xy)$ La matrice $J_{g \circ f}(t) = J_g(f(t))J_f(t)$. $J_g(f(t)) = J_g(t^2, e^t) = ((e^t)^2 \quad 2(t^2)(e^t)) = (e^{2t} \quad 2t^2e^t)$. Le produit matriciel :

$$J_{g \circ f}(t) = (e^{2t} \quad 2t^2e^t) \begin{pmatrix} 2t \\ e^t \end{pmatrix} = 2te^{2t} + (2t^2e^t)(e^t) = 2te^{2t} + 2t^2e^{2t} = 2te^{2t}(1 + t)$$

Les deux méthodes coïncident parfaitement. ■

Exercice 5 : Équation aux dérivées partielles via Chain Rule ★★ ★

★★

Énoncé

Soit $F : \mathbb{R}^2 \rightarrow \mathbb{R}$ une fonction différentiable. On pose $f(r, \theta) = F(r \cos \theta, r \sin \theta)$. Montrer l'identité :

$$\|\nabla F(x, y)\|^2 = \left(\frac{\partial f}{\partial r}\right)^2 + \frac{1}{r^2} \left(\frac{\partial f}{\partial \theta}\right)^2$$

Correction Détaillée

Posons $x(r, \theta) = r \cos \theta$ et $y(r, \theta) = r \sin \theta$. Par la règle de la chaîne :

$$\frac{\partial f}{\partial r} = \frac{\partial F}{\partial x} \frac{\partial x}{\partial r} + \frac{\partial F}{\partial y} \frac{\partial y}{\partial r} = \frac{\partial F}{\partial x} \cos \theta + \frac{\partial F}{\partial y} \sin \theta$$

$$\frac{\partial f}{\partial \theta} = \frac{\partial F}{\partial x} \frac{\partial x}{\partial \theta} + \frac{\partial F}{\partial y} \frac{\partial y}{\partial \theta} = \frac{\partial F}{\partial x} (-r \sin \theta) + \frac{\partial F}{\partial y} (r \cos \theta)$$

Évaluons le membre de droite : $\left(\frac{\partial f}{\partial r}\right)^2 = \left(\frac{\partial F}{\partial x}\right)^2 \cos^2 \theta + \left(\frac{\partial F}{\partial y}\right)^2 \sin^2 \theta + 2 \frac{\partial F}{\partial x} \frac{\partial F}{\partial y} \sin \theta \cos \theta$ $\frac{1}{r^2} \left(\frac{\partial f}{\partial \theta}\right)^2 = \left(-\frac{\partial F}{\partial x} \sin \theta + \frac{\partial F}{\partial y} \cos \theta\right)^2 = \left(\frac{\partial F}{\partial x}\right)^2 \sin^2 \theta + \left(\frac{\partial F}{\partial y}\right)^2 \cos^2 \theta - 2 \frac{\partial F}{\partial x} \frac{\partial F}{\partial y} \sin \theta \cos \theta$ En sommant les deux expressions, les termes croisés s'annulent :

$$\begin{aligned} \left(\frac{\partial f}{\partial r}\right)^2 + \frac{1}{r^2} \left(\frac{\partial f}{\partial \theta}\right)^2 &= \left(\frac{\partial F}{\partial x}\right)^2 (\cos^2 \theta + \sin^2 \theta) + \left(\frac{\partial F}{\partial y}\right)^2 (\sin^2 \theta + \cos^2 \theta) \\ &= \left(\frac{\partial F}{\partial x}\right)^2 + \left(\frac{\partial F}{\partial y}\right)^2 = \|\nabla F(x, y)\|^2 \end{aligned}$$

L'identité est démontrée. ■

Exercice 6 : Différentielle d'une application bilinéaire ★★☆☆

Énoncé

Soit $B : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^m$ une application bilinéaire. Démontrer en utilisant la définition par développement limité que sa différentielle en un point $a = (x_0, y_0)$ appliquée à un vecteur $h = (h_x, h_y)$ est :

$$dB_{(x_0, y_0)}(h_x, h_y) = B(x_0, h_y) + B(h_x, y_0)$$

Correction Détaillée

Calculons $B(a + h) = B(x_0 + h_x, y_0 + h_y)$. Par bilinéarité de B :

$$B(x_0 + h_x, y_0 + h_y) = B(x_0, y_0 + h_y) + B(h_x, y_0 + h_y)$$

$$B(x_0 + h_x, y_0 + h_y) = B(x_0, y_0) + B(x_0, h_y) + B(h_x, y_0) + B(h_x, h_y)$$

Analysons les termes : - $B(x_0, y_0) = B(a)$ (la constante). - L'application $L : (h_x, h_y) \mapsto B(x_0, h_y) + B(h_x, y_0)$ est linéaire par rapport à $h = (h_x, h_y)$. - Le terme de reste est $R(h) = B(h_x, h_y)$. Montrons que $R(h) = o(\|h\|)$. Puisque B est bilinéaire sur des espaces de dimension finie, elle est continue. Il existe $C > 0$ tel que $\|B(u, v)\| \leq C\|u\|\|v\|$. $\|R(h)\| = \|B(h_x, h_y)\| \leq C\|h_x\|\|h_y\|$. Or $\|h_x\| \leq \|h\|$ et $\|h_y\| \leq \|h\|$ (où $\|h\| = \max(\|h_x\|, \|h_y\|)$ par exemple). Donc $\|R(h)\| \leq C\|h\|^2$. Par conséquent, $\frac{\|R(h)\|}{\|h\|} \leq C\|h\| \rightarrow 0$ quand $h \rightarrow 0$. La différentielle est donc bien l'application linéaire L . ■

Exercice 7 : Application de la Chain Rule à une fonction matricielle

★★★☆☆

Énoncé

Soit $f : \mathcal{M}_n(\mathbb{R}) \rightarrow \mathcal{M}_n(\mathbb{R})$ définie par $f(X) = X^2$. Soit $g : \mathcal{M}_n(\mathbb{R}) \rightarrow \mathbb{R}$ définie par $g(X) = \text{Tr}(X)$. Calculer la différentielle de $h = g \circ f$ en A .

Correction Détaillée

Calculons d'abord les différentielles séparément. 1. Différentielle de $f(X) = X^2$: $f(A+H) = (A+H)^2 = A^2 + AH + HA + H^2$. La partie linéaire en H est $df_A(H) = AH + HA$, avec reste $\|H^2\| = O(\|H\|^2)$. 2. Différentielle de $g(X) = \text{Tr}(X)$: Comme la trace est linéaire, sa différentielle est elle-même : $dg_B(K) = \text{Tr}(K)$. 3. Règle de la chaîne : $dh_A(H) = dg_{f(A)}(df_A(H)) = \text{Tr}(df_A(H)) = \text{Tr}(AH + HA)$. Par linéarité de la trace : $\text{Tr}(AH + HA) = \text{Tr}(AH) + \text{Tr}(HA)$. Par propriété de symétrie cyclique : $\text{Tr}(HA) = \text{Tr}(AH)$. Donc $dh_A(H) = 2\text{Tr}(AH)$. La différentielle, au sens scalaire (gradient matriciel), correspond à la matrice $\nabla h(A) = 2A^T$. ■

Exercice 8 : Dérivation d'une équation implicite ★★★☆☆

Énoncé

Soit l'équation $x^2y + y^3x - 2 = 0$. On suppose que cette équation définit localement y comme une fonction de x , soit $y = \phi(x)$. Calculer $\phi'(x)$ en utilisant le théorème de composition.

Correction Détaillée

Posons $F(x, y) = x^2y + y^3x - 2$. Nous savons que $F(x, \phi(x)) = 0$ pour tout x . On définit $g(x) = (x, \phi(x))^T$, de sorte que la fonction composée $h(x) = F(g(x)) = 0$. Par la règle de la chaîne, $dh_x = dF_{g(x)} \circ dg_x = 0$. En termes de matrices jacobienes : $J_h(x) = J_F(x, \phi(x)) \times J_g(x) = 0$. - $J_F(x, y) = \begin{pmatrix} \frac{\partial F}{\partial x} & \frac{\partial F}{\partial y} \end{pmatrix} = (2xy + y^3 \quad x^2 + 3y^2x)$. - $J_g(x) = \begin{pmatrix} 1 \\ \phi'(x) \end{pmatrix}$.

Le produit donne l'équation scalaire :

$$(2x\phi(x) + \phi(x)^3) \cdot 1 + (x^2 + 3\phi(x)^2x) \cdot \phi'(x) = 0$$

On isole $\phi'(x)$:

$$\phi'(x) = -\frac{2x\phi(x) + \phi(x)^3}{x^2 + 3x\phi(x)^2}$$

■

Exercice 9 : Backpropagation manuelle sur un perceptron simple

★★★★★

Énoncé

Considérons une fonction $L(w) = \frac{1}{2}(\sigma(wx) - y)^2$ où $x, y, w \in \mathbb{R}$ et σ est la fonction sigmoïde. Exprimer le gradient $\frac{\partial L}{\partial w}$ à l'aide de la Règle de la Chaîne.

Correction Détaillée

Décomposons la fonction d'erreur $L(w)$ en une suite d'opérations élémentaires : 1. $z = wx$ (transformation affine) 2. $a = \sigma(z)$ (activation) 3. $L = \frac{1}{2}(a - y)^2$ (coût)

Appliquons la règle de la chaîne en remontant de la sortie vers l'entrée (Backpropagation) :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Calculons chaque dérivée locale : - $\frac{\partial L}{\partial a} = a - y$ - $\frac{\partial a}{\partial z} = \sigma'(z) = \sigma(z)(1 - \sigma(z))$ (propriété classique de la sigmoïde) - $\frac{\partial z}{\partial w} = x$

En multipliant le tout :

$$\frac{\partial L}{\partial w} = (a - y) \cdot \sigma'(wx) \cdot x = (\sigma(wx) - y)\sigma(wx)(1 - \sigma(wx))x$$

Ceci est la formule d'apprentissage exacte pour mettre à jour le poids w via descente de gradient.

■

Exercice 10 : Jacobienne d'un réseau multi-couches ★★★★★

Énoncé

Soit un réseau $f(x) = W_2\sigma(W_1x)$ où $x \in \mathbb{R}^n$, $W_1 \in \mathcal{M}_{p,n}(\mathbb{R})$, $W_2 \in \mathcal{M}_{m,p}(\mathbb{R})$ et σ agit composante par composante. Démontrer que la jacobienne de f par rapport à l'entrée x s'écrit : $J_f(x) = W_2\Sigma'(W_1x)W_1$ où $\Sigma'(z)$ est la matrice diagonale des dérivées des activations.

Correction Détaillée

Soit $h = g \circ k \circ l$, avec : - $l(x) = W_1x$: application linéaire, $J_l(x) = W_1$. - $k(z) = \sigma(z)$: application de $\mathbb{R}^p \rightarrow \mathbb{R}^p$ où $k_i(z) = \sigma(z_i)$. Puisque la i -ème coordonnée de k ne dépend que de z_i , les dérivées croisées $\frac{\partial k_i}{\partial z_j}$ sont nulles pour $i \neq j$. Donc $J_k(z)$ est une matrice diagonale : $J_k(z) = \text{Diag}(\sigma'(z_1), \dots, \sigma'(z_p)) = \Sigma'(z)$. - $g(a) = W_2a$: application linéaire, $J_g(a) = W_2$.

Appliquons la règle de la chaîne généralisée :

$$J_f(x) = J_g(k(l(x))) \times J_k(l(x)) \times J_l(x)$$

En remplaçant par les matrices trouvées :

$$J_f(x) = W_2 \times \Sigma'(W_1x) \times W_1$$

Ce produit $m \times p \times p \times p \times n$ donne bien une matrice de taille $m \times n$, reflétant fidèlement l'étirement global de l'espace par le réseau au voisinage de x . ■

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

TP 1 : Calcul symbolique de la Jacobienne

Objectif Mathématique

Utiliser une bibliothèque de calcul formel (SymPy) pour extraire analytiquement la matrice Jacobienne d'un système d'équations multivariées.

Code Python (From Scratch)

```
1 import sympy as sp
2
3 def calculer_jacobienne_analytique():
4     # Declaration des variables symboliques
5     x, y, z = sp.symbols('x y z')
6
7     # Definition des fonctions du systeme
8     f1 = x**2 * y + z
9     f2 = sp.sin(x*y) + z**2
10    f3 = sp.exp(x + y + z)
11
12    F = sp.Matrix([f1, f2, f3])
13    V = sp.Matrix([x, y, z])
14
15    # Calcul direct via la methode jacobian
16    J = F.jacobian(V)
17
18    return J, V
19
20 if __name__ == '__main__':
21     J, V = calculer_jacobienne_analytique()
22     print("Matrice Jacobienne J(x, y, z) :")
23     print(J)
24
25     # Evaluation en un point precis a = (0, 0, 0)
26     J_eval = J.subs({V[0]: 0, V[1]: 0, V[2]: 0})
27     print("\nJacobienne en (0, 0, 0) :")
28     print(J_eval)
29
30     # Assertions mathematiques pour valider les coefficients partiels
31     assert J[0, 0] == 2*V[0]*V[1], "Derivee de f1 par rapport a x
32         incorrecte"
33     assert J_eval[2, 2] == 1, "e^(0+0+0) = 1"
```

TP 2 : Règle de la chaîne numérique

Objectif Mathématique

Implémenter la règle de la chaîne analytique et la comparer avec l'approximation par différences finies centrales sur une composition $g \circ f$.

Code Python (From Scratch)

```
1 import math
2
3 def f(x):
4     return (x**2, math.exp(x))
5
6 def df(x):
7     return (2*x, math.exp(x))
8
9 def g(u, v):
10    return u * v**2
11
12 def dg(u, v):
13    return (v**2, 2*u*v)
14
15 def chain_rule_analytique(x):
16    u, v = f(x)
17    du_dx, dv_dx = df(x)
18    dg_du, dg_dv = dg(u, v)
19
20    # Règle de la chaîne (produit scalaire Jacobienne 1x2 * Jacobienne 2
21    # x1)
22    return dg_du * du_dx + dg_dv * dv_dx
23
24 def differences_finies_centrees(x, epsilon=1e-5):
25    # Approximation (g(f(x+eps)) - g(f(x-eps))) / (2eps)
26    h_plus = g(*f(x + epsilon))
27    h_minus = g(*f(x - epsilon))
28    return (h_plus - h_minus) / (2 * epsilon)
29
30 if __name__ == '__main__':
31    x0 = 1.5
32
33    val_analytique = chain_rule_analytique(x0)
34    val_numerique = differences_finies_centrees(x0)
35
36    print(f"Derivee analytique (Chain Rule) : {val_analytique}")
37    print(f"Derivee numerique (Differences) : {val_numerique}")
38
39    assert abs(val_analytique - val_numerique) < 1e-6, "Divergence entre
40    l'analytique et le numerique"
```


TP 3 : Gradient vectoriel et Rétropropagation

Objectif Mathématique

Simuler le calcul de rétropropagation (Backpropagation) manuellement sur une architecture 1-couche pour voir la Jacobienne en action.

Code Python (From Scratch)

```
1 def forward_pass(w, x, b):
2     z = w * x + b
3     a = 1 / (1 + 2.718281828459**(-z)) # Sigmoid
4     return z, a
5
6 def backpropagation(w, x, b, y_true):
7     z, a = forward_pass(w, x, b)
8
9     # L = 0.5 * (a - y_true)^2
10    # dL/da :
11    dL_da = a - y_true
12
13    # da/dz (Jacobienne de sigmoid) :
14    da_dz = a * (1 - a)
15
16    # dz/dw et dz/db :
17    dz_dw = x
18    dz_db = 1
19
20    # Regle de la chaîne (Produits de derivees)
21    dL_dw = dL_da * da_dz * dz_dw
22    dL_db = dL_da * da_dz * dz_db
23
24    return dL_dw, dL_db
25
26 if __name__ == '__main__':
27     # Initialisation
28     w, b = 0.5, 0.1
29     x, y_true = 2.0, 1.0
30
31     grad_w, grad_b = backpropagation(w, x, b, y_true)
32
33     print(f"Gradient par rapport a w : {grad_w}")
34     print(f"Gradient par rapport a b : {grad_b}")
35
36     # Verification empirique du signe (si a < y_true, on veut augmenter
37     # w (x positif), donc grad_w < 0)
38     z, a = forward_pass(w, x, b)
39     if a < y_true:
40         assert grad_w < 0, "Le gradient devrait etre negatif pour
41                             encourager l'augmentation du poids"
```

TP 4 : Jacobienne géométrique (changement de variable intégral)

Objectif Mathématique

Calculer numériquement le déterminant de la matrice jacobienne (Le Jacobien) pour le changement de variables en coordonnées polaires, validant la mesure d'intégration $rdrd\theta$.

Code Python (From Scratch)

```
1 import math
2
3 def jacobienne_polaires(r, theta):
4     #  $f(r, t) = (r \cos(t), r \sin(t))$ 
5     dx_dr = math.cos(theta)
6     dx_dt = -r * math.sin(theta)
7
8     dy_dr = math.sin(theta)
9     dy_dt = r * math.cos(theta)
10
11     return [[dx_dr, dx_dt], [dy_dr, dy_dt]]
12
13 def determinant_2x2(matrix):
14     return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
15
16 if __name__ == '__main__':
17     points_test = [
18         (1.0, 0.0),
19         (2.5, math.pi / 4),
20         (5.0, math.pi / 2)
21     ]
22
23     for r, theta in points_test:
24         J = jacobienne_polaires(r, theta)
25         det_J = determinant_2x2(J)
26
27         print(f"Rayon r={r}, Theta={theta:.2f} -> Det(J)={det_J}")
28
29         # Le determinant de la jacobienne polaire DOIT etre exactement r
30         assert abs(det_J - r) < 1e-9, "Le jacobien ne correspond pas a la theorie."
```

TP 5 : Vector-Jacobian Product (VJP)

Objectif Mathématique

L'opération fondamentale des frameworks de Deep Learning modernes (PyTorch, JAX) n'est pas de matérialiser la matrice Jacobienne complète, mais de calculer le produit d'un vecteur par la Jacobienne transposée ($v^T J$). Implémentons ce VJP pour une fonction multi-dimensionnelle.

Code Python (From Scratch)

```
1 def relu(x):
2     return [max(0, xi) for xi in x]
3
4 def vjp_relu(v, x):
5     # La jacobienne de ReLU est diagonale avec 1 si x > 0, 0 sinon.
6     # v^T J revient à faire v_i * (1 si x_i > 0 else 0)
7     return [vi * (1.0 if xi > 0 else 0.0) for vi, xi in zip(v, x)]
8
9 def couche_lineaire(W, x):
10    # y = W * x
11    return [sum(w_ij * x_j for w_ij, x_j in zip(row, x)) for row in W]
12
13 def vjp_lineaire(v, W):
14    # J = W. Donc v^T J = v^T W = W^T v
15    cols = len(W[0])
16    rows = len(W)
17    return [sum(W[i][j] * v[j] for j in range(cols)) for i in range(rows)]
18
19 if __name__ == '__main__':
20     # Exemple : Couche lineaire suivie de ReLU
21     x = [0.5, -1.0, 2.0]
22     W = [
23         [1.0, 0.5, -0.5],
24         [0.0, 1.0, 2.0]
25     ]
26
27     # Passe Forward
28     z = couche_lineaire(W, x)
29     a = relu(z)
30
31     # Supposons un vecteur gradient venant de la fin du reseau
32     grad_a = [0.1, -0.2]
33
34     # Passe Backward (VJP successifs, de l'exterieur vers l'interieur)
35     grad_z = vjp_relu(grad_a, z)
36     grad_x = vjp_lineaire(grad_z, W)
37
38     print("Vecteur d'entree x:", x)
39     print("Gradient reçu grad_a:", grad_a)
40     print("Gradient propage grad_x:", grad_x)
41
42     # Validation dimensionnelle
43     assert len(grad_x) == len(x), "Le VJP doit conserver la dimension de l'espace de depart."
```